

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра Програмної інженерії

Звіт
з практичної роботи №2
з дисципліни: «Високорівневі мови програмування та фреймворки»
з теми: «Веб-розробка з JavaScript та React»

Виконав:
ст. гр. ПЗПІ-23-2
Ситник Є. С.
Варіант: №12

Перевірів:
ст. викл. кафедри ПІ
Саманцов О. О.

2 ВЕБ-РОЗРОБКА З JAVASCRIPT ТА REACT

2.1 Мета роботи

Метою даної роботи є ознайомлення з форматом даних JSON, робота із запитамися через метод `fetch()` та основами розробки на React. Розгляд принципів взаємодії клієнтської частини з API, парсингу структурованих даних та створення інтерактивних вебсторінок.

2.2 Індивідуальне завдання

а) Рівень 1: Список покупок:

- 1) Створіть файл HTML з елементом `<script>`.
- 2) Використовуйте `fetch` для завантаження JSON-файлу зі списком покупок.
- 3) Розберіть JSON-файл за допомогою `JSON.parse()`.
- 4) Створіть список HTML з елементами `<li>` для кожного пункту списку покупок.

б) Рівень 2: Фільтр JSON.

- 1) Напишіть програму, яка приймає JSON-файл та значення фільтра в якості вхідних даних.
- 2) Програма повинна фільтрувати дані JSON, залишаючи лише ті елементи, які відповідають заданому значенню фільтра.
- 3) Виведіть відфільтровані дані JSON.

в) Рівень 3:

- 1) Розробіть веб-додаток для обміну повідомленнями між користувачами.
- 2) Додайте можливість створення чат-кімнат та відправлення повідомлень.

г) Рівень 4: Додайте можливість фільтрації, сортування і пошуку.

2.3 Хід роботи

2.3.1 Рівень 1.

```
<meta charset="UTF-8">

<h3>Мій список покупок:</h3>
<ul id="list"></ul>

<script>
  fetch('data.json')
    .then(response => response.text())
    .then(text => {
      const products = JSON.parse(text);
      list.innerHTML = products
        .map(item => `<li>${item}</li>`)
        .join('');
    });
</script>
```

2.3.2 Рівень 2.

```
<meta charset="UTF-8">

<input type="text" id="search" placeholder="Фільтр"
oninput="filterData()">
<ul id="list"></ul>

<script>
  let products = [];

  fetch('data.json')
    .then(res => res.json())
    .then(data => {
      products = data
      filterData()
    });

  function filterData() {
    const query = search.value.toLowerCase()
    const filtered = products.filter(item =>
      item.name.toLowerCase().includes(query) ||
      item.category.toLowerCase().includes(query)
    );

    list.innerHTML = filtered
      .map(item => `<li>${item.name} (${item.category})</li>`)
      .join('')
  }
</script>
```

2.3.3 Підбірка 3 та 4

2.3.3.1 index.html

```
<!DOCTYPE html>
<html lang="uk">
<head>
  <meta charset="UTF-8">
  <title>Chat</title>
</head>
<body>
  <div id="root"></div>
  <script src="/app.js"></script>
</body>
</html>
```

2.3.3.2 app.jsx

```
import React, { useState, useEffect } from 'react';
import { createRoot } from 'react-dom/client';

function App() {
  const [rooms, setRooms] = useState([]);
  const [messages, setMessages] = useState([]);
  const [activeRoom, setActiveRoom] = useState('Головна');
  const [newRoomName, setNewRoomName] = useState('');
  const [newMessageText, setNewMessageText] = useState('');
  const [search, setSearch] = useState('');
  const [sort, setSort] = useState('oldest');
  const [myName] = useState(() => prompt("Введіть ваше ім'я:") || "Анонім");

  const fetchRooms = () => fetch('/api/rooms').then(r =>
r.json()).then(setRooms);

  const fetchMessages = () => {
    fetch(`/api/messages?room=${activeRoom}&search=${search}
&sort=${sort}`)
      .then(r => r.json())
      .then(setMessages);
  };

  useEffect(() => {
    fetchRooms();
    fetchMessages();

    const timerId = setInterval(() => { fetchMessages() }, 3000);

    return () => clearInterval(timerId);
  }, [activeRoom, search, sort]);

  const createRoom = () => {
    if (!newRoomName) return;
    fetch('/api/rooms', {
```

```

        method: 'POST',
        body: JSON.stringify({ name: newRoomName })
    }).then(() => { setNewRoomName(''); fetchRooms(); });
};

const sendMessage = () => {
    if (!newMessageText) return;
    fetch('/api/messages', {
        method: 'POST',
        body: JSON.stringify({ room: activeRoom, text:
newMessageText, sender: myName })
    }).then(() => { setNewMessageText(''); fetchMessages(); });
};

return (
    <div style={{ padding: '20px', fontFamily: 'sans-serif' }}>
        <h3>Кімнати:</h3>
        <input value={newRoomName} onChange={e =>
setNewRoomName(e.target.value)} placeholder="Нова кімната..." />
        <button onClick={createRoom}>Створити</button>
        <ul>
            {rooms.map(room => (
                <li key={room}>
                    <button onClick={() => setActiveRoom(room)}>
                        {room === activeRoom ? '👉' : ''}{room}
                    </button>
                </li>
            ))}
        </ul>
        <hr />

        <input value={search} onChange={e =>
setSearch(e.target.value)} placeholder="Пошук..." />
        <select value={sort} onChange={e =>
setSort(e.target.value)}>
            <option value="oldest">Спочатку старі</option>
            <option value="newest">Спочатку нові</option>
        </select>
        <br /><br />

        <ul>
            {messages.map((msg, idx) => <li key={idx}
><b>{msg.sender}</b>: </b> {msg.text}</li>)}
        </ul>

        <input
            value={newMessageText}
            onChange={e => setNewMessageText(e.target.value)}
            placeholder="Повідомлення..."
            onKeyDown={e => e.key === 'Enter' && sendMessage()}
        />
        <button onClick={sendMessage}>Відправити</button>
    </div>
);
}

```

```
const root = createRoot(document.getElementById('root'));
root.render(<App />);
```

2.3.3.3 server.js

```
import { Database } from "bun:sqlite";

const db = new Database("chat.sqlite");
db
  .query("CREATE TABLE IF NOT EXISTS rooms (name TEXT UNIQUE)")
  .run();
db
  .query("CREATE TABLE IF NOT EXISTS messages (room TEXT, text TEXT,
sender TEXT, time INTEGER)")
  .run();

try {
  db
    .query("INSERT INTO rooms (name) VALUES ('Головна')")
    .run();
} catch { }

await Bun.build({
  entrypoints: ['./app.jsx'],
  outdir: './build',
});

Bun.serve({
  port: 6969,
  async fetch(req) {
    const url = new URL(req.url);
    const method = req.method;

    if (url.pathname === "/")
      return new Response(Bun.file("index.html"));

    if (url.pathname === "/app.js")
      return new Response(Bun.file("build/app.js"));

    if (url.pathname === "/api/rooms" && method === "GET") {
      const rooms = db.query("SELECT name FROM rooms").all().map(r =>
r.name);
      return Response.json(rooms);
    }

    if (url.pathname === "/api/rooms" && method === "POST") {
      const { name } = await req.json();
      try { db.query("INSERT INTO rooms (name) VALUES
(?)").run(name); } catch { }
      return Response.json({ success: true });
    }

    if (url.pathname === "/api/messages" && method === "GET") {
      const room = url.searchParams.get("room") || "Головна";
      const search = url.searchParams.get("search") || "";
      const sort = url.searchParams.get("sort") || "oldest";
```

```

    const order = sort === "newest" ? "DESC" : "ASC";

    const msgs = db
      .query(`SELECT * FROM messages WHERE room = ? AND text LIKE ?
ORDER BY time ${order}`)
      .all(room, `%${search}%`);

    return Response.json(msgs);
  }

  if (url.pathname === "/api/messages" && method === "POST") {
    const { room, text, sender } = await req.json();

    db
      .query("INSERT INTO messages (room, text, sender, time)
VALUES (?, ?, ?, ?)")
      .run(room, text, sender, Date.now());

    return Response.json({ success: true });
  }

  return new Response("Not found", { status: 404 });
});

```

2.4 Висновки

Під час виконання даної роботи було розглянуто роботу із форматом JSON та виконання мережевих запитів методом `fetch()`. Реалізовано відображення даних на сторінці, роботу з API, а також розроблено функціональні компоненти React для управління списками, фільтрації та взаємодії з користувачем.

ДОДАТОК А

Дані для перевірки роботи завдань рівнів 1 та 2

```
[
  {
    "name": "Яблуко",
    "category": "фрукти"
  },
  {
    "name": "Морква",
    "category": "овочі"
  },
  {
    "name": "Банан",
    "category": "фрукти"
  },
  {
    "name": "Помідор",
    "category": "овочі"
  }
]
```